

RAGAS 这四个指标可以理解为在评估 RAG 系统的四个环节：

| 指标                          | 主要评价对象       | 核心问题                 | 通俗说法       |
|-----------------------------|--------------|----------------------|------------|
| Context Precision           | 检索器排序质量      | 找回来的材料里，相关材料是不是排在前面？ | 好东西有没有放前面  |
| Context Recall              | 检索器覆盖率       | 应该找回的关键信息，有没有漏掉？     | 该找的有没有找全   |
| Faithfulness                | 生成答案是否忠实于上下文 | 答案里的事实，是否都能被检索材料支持？  | 有没有编、有没有幻觉 |
| Answer / Response Relevancy | 生成答案是否回应问题   | 答案是否直接回答了用户问题？       | 有没有答非所问    |

## 1. Context Precision：检索结果“排得好不好”

### 核心内容

Context Precision 衡量的是：对于一个用户问题，检索器返回了一组 contexts / chunks，其中真正有用的内容是否排在更靠前的位置。它关注的不是“有没有找全”，而是“找回来的内容排序是否合理”。RAGAS 文档中说明，它会看相关 chunk 是否位于 retrieved contexts 的顶部，并基于 precision@k 计算分数。

通俗理解：

你问“埃菲尔铁塔在哪里？”，系统检索出两段材料：

- 埃菲尔铁塔位于巴黎。
- 勃兰登堡门位于柏林。

这就很好，因为有用材料排在第一位。

但如果顺序变成：

- 勃兰登堡门位于柏林。
- 埃菲尔铁塔位于巴黎。

虽然答案材料也被找到了，但排序不好，Context Precision 会下降。附件中的例子也说明：无关 chunk 放在第二位影响较小，但如果放在第一位，分数会明显降低。

### 它在 RAG 系统里的意义

Context Precision 主要看 **retriever 的排序能力**。

在真实 RAG 中，LLM 通常优先使用前几个 chunk。如果相关信息排得太靠后，即使被检索出来，也可能被模型忽略，或者被噪声干扰。

## 适合用在什么时候

当你想判断：

“我的向量检索、重排序、BM25 + embedding hybrid search，到底有没有把最相关内容排在前面？”

就看 Context Precision。

---

## 2. Context Recall：检索结果“找得全不全”

### 核心内容

Context Recall 衡量的是：所有应该被检索出来的相关信息中，系统实际找回了多少。RAGAS 文档强调，Recall 关注的是“不遗漏重要信息”，高 recall 意味着更少的相关文档或关键信息被漏掉。它通常需要 reference 作为对照，把参考答案拆成多个 claim，再判断这些 claim 是否能被 retrieved context 支持。

公式可以理解为：

Context Recall = 被检索上下文支持的 reference claims 数量 / reference 中的全部 claims 数量

通俗理解：

假设标准答案是：

“埃菲尔铁塔位于巴黎，由古斯塔夫·埃菲尔设计，建于 1889 年。”

这里有三个关键信息：

1. 位于巴黎
2. 由古斯塔夫·埃菲尔设计
3. 建于 1889 年

如果检索结果只找到了“埃菲尔铁塔位于巴黎”，那 recall 就不高，因为它只覆盖了部分必要信息。

### 它和 Context Precision 的区别

Precision 是“排得准不准”，Recall 是“找得全不全”。

一个系统可能：

- Precision 高：前几条都很相关；
- Recall 低：但漏掉了 many 必要信息。

也可能：

- Recall 高：相关信息都找到了；
- Precision 低：但混在大量无关信息里，排序很差。

## 适合用在什么时候

当你想判断：

“我的知识库检索是否漏掉了关键证据？用户问复杂问题时，必要材料有没有被召回？”

就看 Context Recall。

---

## 3. Faithfulness：答案“有没有忠实依据材料”

### 核心内容

Faithfulness 衡量的是生成回答与 retrieved context 的事实一致性。RAGAS 文档中说，如果回答中的所有 claims 都能被检索上下文支持，那么这个回答就是 faithful。计算时会先识别 response 中的 claims，再逐一判断这些 claims 是否能从 retrieved context 中推出，最后计算被支持 claim 的比例。

通俗理解：

Context 是：

“爱因斯坦出生于 1879 年 3 月 14 日，是德国出生的理论物理学家。”

如果回答是：

“爱因斯坦于 1879 年 3 月 14 日出生在德国。”

这就是高 Faithfulness。

但如果回答是：

“爱因斯坦于 1879 年 3 月 20 日出生在德国。”

这里“出生在德国”能被支持，但“3 月 20 日”不能被支持，所以 Faithfulness 下降。附件中的示例将该回答拆成两个 statement，其中一个支持、一个不支持，因此分数为 0.5。

## 它在 RAG 系统里的意义

Faithfulness 是最接近“幻觉检测”的指标。

它不是问答案是否真的符合世界事实，而是问：

“这个答案是否忠实于你给它的上下文？”

这点很重要。即使答案在现实世界中是对的，但如果 retrieved context 没有提供依据，Faithfulness 也可能不高。因为 RAG 的目标是“基于证据回答”，而不是凭模型记忆回答。

## 适合用在什么时候

当你想判断：

“LLM 有没有根据检索材料回答？有没有自己脑补？”

就看 Faithfulness。

---

## 4. Answer / Response Relevancy：答案“有没有答到点上”

### 核心内容

Response Relevancy，也叫 Answer Relevancy，衡量的是回答和用户问题的相关程度。附件中说明，它不评估事实准确性，而是关注回答是否直接、适当地回应了原始问题；不完整回答或包含不必要细节的回答会被惩罚。

它的计算方式比较有意思：

1. 根据模型生成的回答，反向生成若干个“可能的问题”；
2. 把这些反向生成的问题和原始用户问题做 embedding；
3. 计算两者的余弦相似度；
4. 平均后得到 Answer Relevancy 分数。

通俗理解：

用户问：

“法国在哪里？它的首都是哪里？”

回答 A：

“法国位于西欧。”

回答 B：

“法国位于西欧，首都是巴黎。”

回答 A 不是错，但只回答了一半，所以相关性不够高。回答 B 同时回应了“在哪里”和“首都是哪里”，所以 relevancy 更高。附件也用了类似例子说明低相关和高相关回答的区别。

## 它和 Faithfulness 的区别

Faithfulness 看的是：

“答案有没有被材料支持？”

Response Relevancy 看的是：

“答案有没有回答用户的问题？”

一个答案可以很 faithful，但不 relevant。

例如用户问“苹果公司的 CEO 是谁？”，检索材料里有“苹果公司总部在库比蒂诺”，模型回答“苹果公司总部在库比蒂诺”。这句话可能忠实于材料，但没有回答 CEO 问题，所以 relevancy 低。

一个答案也可以很 relevant，但不 faithful。

例如用户问“埃菲尔铁塔在哪里？”，模型回答“埃菲尔铁塔在巴黎”。这很相关，但如果 retrieved context 里没有任何支持材料，Faithfulness 可能不高。

---

## 四个指标放在 RAG 流程中的位置

可以把 RAG 过程拆成三步：

用户问题 → 检索器找资料 → LLM 基于资料生成答案

对应关系是：

检索器找资料：

- Context Recall: 有没有找全？
- Context Precision: 有没有把有用资料排前面？

LLM 生成答案：

- Faithfulness: 有没有忠实使用资料？
- Response Relevancy: 有没有回答用户问题？

所以这四个指标不是互相替代，而是分别定位问题。

---

## 最实用的诊断方式

如果你的 RAG 回答效果不好，可以这样排查：

| 现象           | 可能问题         | 优先看哪个指标                  |
|--------------|--------------|--------------------------|
| 答案缺少关键信息     | 检索漏掉关键材料     | Context Recall           |
| 答案被无关材料干扰    | 检索排序差、噪声靠前   | Context Precision        |
| 答案看起来很完整但有编造 | 生成阶段幻觉       | Faithfulness             |
| 答案事实没错但没回答问题 | 回答偏题或不完整     | Response Relevancy       |
| 找到了材料但模型没用好  | 生成器没有充分利用上下文 | Faithfulness + Relevancy |
| 材料很多但答案质量差   | 检索噪声高、排序差    | Context Precision        |

## 这四个指标的本质差别

最简单的区分方式是：

**Context Precision 和 Context Recall 是在评估“资料准备得怎么样”。**

前者问：

你给模型看的资料，前面是不是有用的？

后者问：

你该给模型看的资料，有没有漏？

**Faithfulness 和 Response Relevancy 是在评估“答案写得怎么样”。**

前者问：

答案有没有根据资料说话？

后者问：

答案有没有围绕用户问题说话？

所以在搭建 RAG 系统时，建议不要只看一个总分，而是四个指标一起看。因为 RAG 的失败可能发生在检索阶段，也可能发生在生成阶段。Context Precision / Recall 能帮你定位 retriever 问题，Faithfulness / Relevancy 能帮你定位 generator 问题。

## 5、评估指标的具体方式

真正落地时通常要写脚本测试，但不是一上来就写复杂评估平台，而是先做一个很小的 **eval dataset + RAGAS 脚本**，把每次 RAG 系统的输入、检索结果、生成答案和参考答案记录下来，然后批量算分。

你可以把它理解成：  
**RAGAS 不是直接“看线上系统感觉好不好”，而是给一批标准问题跑自动化打分。**

### 1) 你需要准备什么数据？

最小评估样本一般长这样：

| 字段                 | 含义                   | 哪些指标会用                                                |
|--------------------|----------------------|-------------------------------------------------------|
| user_input         | 用户问题                 | 四个指标都需要                                               |
| retrieved_contexts | RAG 检索出来的 chunks     | Context Precision、Context Recall、Faithfulness         |
| response           | LLM 最终生成的答案          | Faithfulness、Answer Relevancy、部分 Context Precision 变体 |
| reference          | 人工标准答案 / 参考答案        | Context Precision、Context Recall                      |
| reference_contexts | 理想情况下应该检索到的标准 chunks | 非 LLM 版本 Recall / Precision、ID-based 版本               |

最小可运行版本不一定要准备 `reference_contexts`，但最好准备 `reference`。因为 Context Recall 通常需要 `reference` 来对照，RAGAS 会把 `reference` 拆成 `claims`，再判断 `retrieved contexts` 是否支持这些 `claims`。

### 2) 最小可行的测试流程

你可以按这个流程来做：

1. 准备 20~50 个典型问题
  2. 给每个问题人工写 `reference` 标准答案
  3. 用你的 RAG 系统跑这些问题

4. 记录每个问题的 `retrieved_contexts` 和 `response`
5. 用 RAGAS 批量计算四个指标
6. 看均值、低分样本、问题归因

真正重要的不是平均分，而是 **低分样本回放**。比如某条样本：

Context Recall 低：说明关键材料没找回来

Context Precision 低：说明找回来了，但排序靠后或噪声太多

Faithfulness 低：说明模型基于材料之外脑补

Answer Relevancy 低：说明答案没有围绕问题回答

### 3) 一个最小 Python 脚本长什么样？

下面是一个概念版。你的项目里可以把 `retrieved_contexts` 和 `response` 换成真实 RAG 系统输出。

```
import asyncio
import pandas as pd

from openai import AsyncOpenAI
from ragas.llms import llm_factory
from ragas.embeddings.base import embedding_factory
from ragas.metrics.collections import (
    Faithfulness,
    AnswerRelevancy,
    ContextPrecision,
    ContextRecall,
)

client = AsyncOpenAI()

llm = llm_factory("gpt-4o-mini", client=client)
embeddings = embedding_factory(
    "openai",
    model="text-embedding-3-small",
    client=client,
)

faithfulness = Faithfulness(llm=llm)
answer_relevancy = AnswerRelevancy(llm=llm, embeddings=embeddings)
context_precision = ContextPrecision(llm=llm)
context_recall = ContextRecall(llm=llm)
```



```
eval_samples = [
    {
        "user_input": "埃菲尔铁塔在哪里?",
        "retrieved_contexts": [
            "埃菲尔铁塔位于法国巴黎。",
            "勃兰登堡门位于德国柏林。"
        ],
        "response": "埃菲尔铁塔位于法国巴黎。",
        "reference": "埃菲尔铁塔位于巴黎。"
    },
    {
        "user_input": "第一届超级碗是什么时候举办的?",
        "retrieved_contexts": [
            "第一届 AFL-NFL 世界冠军赛于 1967 年 1 月 15 日举行。"
        ],
        "response": "第一届超级碗于 1967 年 1 月 15 日举办。",
        "reference": "第一届超级碗于 1967 年 1 月 15 日举办。"
    },
]
```

```
async def evaluate_one(sample):
    user_input = sample["user_input"]
    retrieved_contexts = sample["retrieved_contexts"]
    response = sample["response"]
    reference = sample["reference"]

    f = await faithfulness.ascore(
        user_input=user_input,
        response=response,
        retrieved_contexts=retrieved_contexts,
    )

    ar = await answer_relevancy.ascore(
        user_input=user_input,
        response=response,
    )

    cp = await context_precision.ascore(
        user_input=user_input,
        reference=reference,
        retrieved_contexts=retrieved_contexts,
    )

    cr = await context_recall.ascore(
        user_input=user_input,
```

```

        reference=reference,
        retrieved_contexts=retrieved_contexts,
    )

    return {
        **sample,
        "faithfulness": f.value,
        "answer_relevancy": ar.value,
        "context_precision": cp.value,
        "context_recall": cr.value,
    }

async def main():
    results = []
    for sample in eval_samples:
        result = await evaluate_one(sample)
        results.append(result)

    df = pd.DataFrame(results)
    print(df[
        [
            "user_input",
            "faithfulness",
            "answer_relevancy",
            "context_precision",
            "context_recall",
        ]
    ])

    print("\nAverage scores:")
    print(df[
        [
            "faithfulness",
            "answer_relevancy",
            "context_precision",
            "context_recall",
        ]
    ].mean())

    df.to_csv("ragas_eval_results.csv", index=False, encoding="utf-8-sig")

if __name__ == "__main__":
    asyncio.run(main())

```

附件里的示例也是这种用法：先创建 metric scorer，然后传入 `user_input` 、`retrieved_contexts` 、`response` 、`reference` 等字段，再调用 `.ascore()` 或同步的

`.score()` 方法。

## 4) 四个指标分别需要哪些字段？

你可以按这个表来记：

| 指标                | 最小字段                                        | 是否需要人工标准答案 |
|-------------------|---------------------------------------------|------------|
| Faithfulness      | user_input 、 response 、 retrieved_contexts  | 不一定        |
| Answer Relevancy  | user_input 、 response                       | 不需要        |
| Context Precision | user_input 、 retrieved_contexts 、 reference | 通常需要       |
| Context Recall    | user_input 、 retrieved_contexts 、 reference | 需要         |

Faithfulness 是把 response 拆成 claims，再看这些 claims 是否被 retrieved context 支持；附件中爱因斯坦生日例子就是典型：两个 claim 中只有一个被 context 支持，所以得分 0.5。

Answer Relevancy 则不看 context，它主要看回答是否贴合问题。RAGAS 的方法是基于 response 反向生成若干问题，再和原始问题做 embedding 相似度比较。

## 5) 具体怎么看分数？

不要机械理解成“0.8 以上就一定好”。更好的方式是分层看。

### 第一层：看平均分

例如：

| 指标                | 平均分  | 解释      |
|-------------------|------|---------|
| Context Precision | 0.82 | 检索排序还可以 |
| Context Recall    | 0.55 | 关键材料经常漏 |
| Faithfulness      | 0.91 | 模型基本不乱编 |
| Answer Relevancy  | 0.88 | 回答基本贴题  |

这个结果说明：

生成模型问题不大，主要问题在检索召回不足。

## 第二层：看低分样本

比如筛选：

```
bad_cases = df[
    (df["context_recall"] < 0.6)
    | (df["faithfulness"] < 0.7)
    | (df["answer_relevancy"] < 0.7)
]
```

然后逐条看：

问题是什么？  
标准答案是什么？  
检索到了哪些 chunks？  
模型最终回答是什么？  
哪个指标低？

这一步最有价值，因为它能告诉你应该改哪里。

## 第三层：按问题类型分组

比如你可以给 eval 样本加一列：

```
question_type:
- factual_lookup 单事实查询
- multi_hop 多跳推理
- summary 总结类
- comparison 对比类
- policy_rule 制度/条款类
```

然后看不同类型的平均分。

如果制度/条款类 Context Recall 低，可能说明 chunk 切分太碎或检索 query 没有覆盖条款同义词。

如果 summary 类 Faithfulness 低，可能说明生成 prompt 约束不够，模型会概括过头。

---

## 6. 对你做 RAG / Agent 项目的建议

我建议你按“最小目标 + 流程链”来做：

**Input:**

20~50 个高质量评估问题

每个问题对应 **reference** 标准答案

**Process:**

跑你的 RAG 系统，保存 **retrieved\_contexts** 和 **response**

用 RAGAS 批量计算四个指标

人工复盘低分样本

**Output:**

一个 **eval\_results.csv**

一个 **bad\_cases.csv**

一份问题归因表

第一版不要追求大而全。

你真正要建立的是一个工程闭环：

改检索策略 / **chunk** 策略 / **prompt**

→ 跑同一批 **eval**

→ 对比分数变化

→ 查看低分样本

→ 决定下一轮优化方向

这就是 RAGAS 最核心的使用方式。它不是“评一次就结束”，而是作为 RAG 系统迭代时的回归测试工具。